



Campus: Polo Ribeiraria

Curso: 7124 - Desenvolvimento Full Stack

Disciplina: DGT2821 - Programação Back-end Com Java

Turma: 9001 **Semestre:** 2026.1

Matrícula: 2025 0324 5401

Aluno: Alexandre Vecchi

1. Título da Prática

1º Procedimento | Criação das Entidades e Sistema de Persistência

2. Objetivo da Prática

1. Utilizar herança e polimorfismo na definição de entidades.
2. Utilizar persistência de objetos em arquivos binários.
3. Implementar uma interface cadastral em modo texto.
4. Utilizar o controle de exceções da plataforma Java.
5. No final do projeto, o aluno terá implementado um sistema cadastral em Java, utilizando os recursos da programação orientada a objetos e a persistência em arquivos binários.

3. Todos os códigos solicitados neste roteiro de aula

1. mainapp.model – Pessoa.java

```
package model;

import java.io.Serializable;

public class Pessoa implements Serializable {

    private int id;
    private String nome;

    public Pessoa() {
    }

    public Pessoa(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public void exibir() {
        System.out.println("Id: " + id);
        System.out.println("Nome: " + nome);
    }
}
```

2. mainapp.model – PessoaFisica.java

```
package model;

import java.io.Serializable;

public class PessoaFisica extends Pessoa implements Serializable {

    private String cpf;
    private int idade;

    public PessoaFisica() {
    }

    public PessoaFisica(int id, String nome, String cpf, int idade) {
        super(id, nome);
        this.cpf = cpf;
        this.idade = idade;
    }

    public String getCpf() {
        return cpf;
    }

    public void setCpf(String cpf) {
        this.cpf = cpf;
    }

    public int getIdade() {
        return idade;
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }

    @Override
    public void exibir() {
        super.exibir();
        System.out.println("CPF: " + cpf);
        System.out.println("Idade: " + idade);
    }
}
```

3. mainapp.model – PessoaFisicaRepo.java

```
package model;

import java.io.*;
import java.util.ArrayList;

public class PessoaFisicaRepo {

    private ArrayList<PessoaFisica> listaPessoas = new ArrayList<>();

    public void inserir(PessoaFisica pessoa) {
        listaPessoas.add(pessoa);
    }

    public void alterar(PessoaFisica pessoaAlterada) {
        for (int i = 0; i < listaPessoas.size(); i++) {
            if (listaPessoas.get(i).getId() == pessoaAlterada.getId()) {

```

```

        listaPessoas.set(i, pessoaAlterada);
        return;
    }
}

public void excluir(int id) {
    listaPessoas.removeIf(p -> p.getId() == id);
}

public PessoaFisica obter(int id) {
    for (PessoaFisica p : listaPessoas) {
        if (p.getId() == id) {
            return p;
        }
    }
    return null;
}

public ArrayList<PessoaFisica> obterTodos() {
    return listaPessoas;
}

public void persistir(String nomeArquivo) throws IOException {
    try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(nomeArquivo))) {
        out.writeObject(listaPessoas);
        System.out.println("Dados de Pessoa Fisica Armazenados.");
    }
}

public void recuperar(String nomeArquivo) throws IOException, ClassNotFoundException {
    try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(nomeArquivo))) {
        listaPessoas = (ArrayList<PessoaFisica>) in.readObject();
        System.out.println("Dados de Pessoa Fisica Recuperados.");
    }
}
}

```

4. mainapp.model – PessoaJuridica.java

```

package model;

import java.io.Serializable;

public class PessoaJuridica extends Pessoa implements Serializable {
    private String cnpj;

    public PessoaJuridica() {
    }

    public PessoaJuridica(int id, String nome, String cnpj) {
        super(id, nome);
        this.cnpj = cnpj;
    }

    @Override
    public void exibir() {
        super.exibir();
        System.out.println("CNPJ: " + cnpj);
    }
}

```

5. mainapp.model – PessoaJuridicaRepo.java

```
package model;

import java.io.*;
import java.util.ArrayList;

public class PessoaJuridicaRepo {

    private ArrayList<PessoaJuridica> listaPessoas = new ArrayList<>();

    public void inserir(PessoaJuridica pessoa) {
        listaPessoas.add(pessoa);
    }

    public void alterar(PessoaJuridica pessoaAlterada) {
        for (int i = 0; i < listaPessoas.size(); i++) {
            if (listaPessoas.get(i).getId() == pessoaAlterada.getId()) {
                listaPessoas.set(i, pessoaAlterada);
                return;
            }
        }
    }

    public void excluir(int id) {
        listaPessoas.removeIf(p -> p.getId() == id);
    }

    public PessoaJuridica obter(int id) {
        for (PessoaJuridica p : listaPessoas) {
            if (p.getId() == id) {
                return p;
            }
        }
        return null;
    }

    public ArrayList<PessoaJuridica> obterTodos() {
        return listaPessoas;
    }

    public void persistir(String nomeArquivo) throws IOException {
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(nomeArquivo))) {
            out.writeObject(listaPessoas);
            System.out.println("Dados de Pessoa Juridica Armazenados.");
        }
    }

    public void recuperar(String nomeArquivo) throws IOException, ClassNotFoundException {
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(nomeArquivo))) {
            listaPessoas = (ArrayList<PessoaJuridica>) in.readObject();
            System.out.println("Dados de Pessoa Juridica Recuperados.");
        }
    }
}
```

6. mainapp.mainapp – MainApp.java

```
package mainapp;

import java.io.IOException;
import java.util.logging.Level;
```

```

import java.util.logging.Logger;
import model.*;

public class MainApp {

    public static void main(String[] args) {

        PessoaFisicaRepo repo1 = new PessoaFisicaRepo();

        repo1.inserir(new PessoaFisica(1, "Ana", "1111111111", 25));
        repo1.inserir(new PessoaFisica(2, "Carlos", "2222222222", 52));

        try {
            repo1.persistir("PessoaFisica.dat");
        } catch (IOException ex) {
            Logger.getLogger(MainApp.class.getName()).log(Level.SEVERE, null, ex);
        }

        PessoaFisicaRepo repo2 = new PessoaFisicaRepo();

        try {
            repo2.recuperar("PessoaFisica.dat");
        } catch (IOException | ClassNotFoundException ex) {
            Logger.getLogger(MainApp.class.getName()).log(Level.SEVERE, null, ex);
        }

        for (PessoaFisica pf : repo2.obterTodos()) {
            pf.exibir();
        }

        PessoaJuridicaRepo repo3 = new PessoaJuridicaRepo();

        repo3.inserir(new PessoaJuridica(3, "XPTO Sales", "3333333333333333"));
        repo3.inserir(new PessoaJuridica(4, "XPTO Solutions", "4444444444444444"));

        try {
            repo3.persistir("PessoaJuridica.dat");
        } catch (IOException ex) {
            Logger.getLogger(MainApp.class.getName()).log(Level.SEVERE, null, ex);
        }

        PessoaJuridicaRepo repo4 = new PessoaJuridicaRepo();

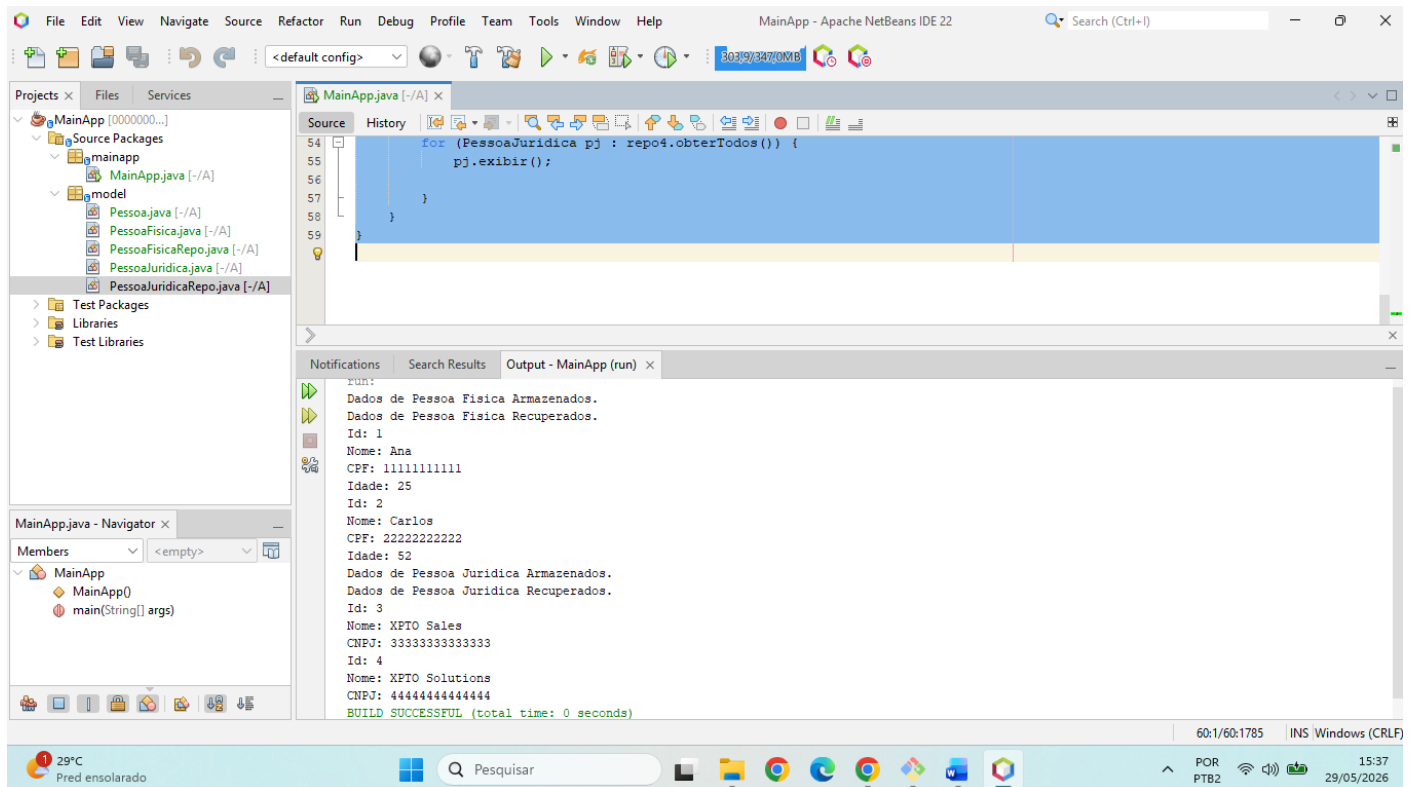
        try {
            repo4.recuperar("PessoaJuridica.dat");
        } catch (IOException | ClassNotFoundException ex) {
            Logger.getLogger(MainApp.class.getName()).log(Level.SEVERE, null, ex);
        }

        for (PessoaJuridica pj : repo4.obterTodos()) {
            pj.exibir();
        }
    }
}

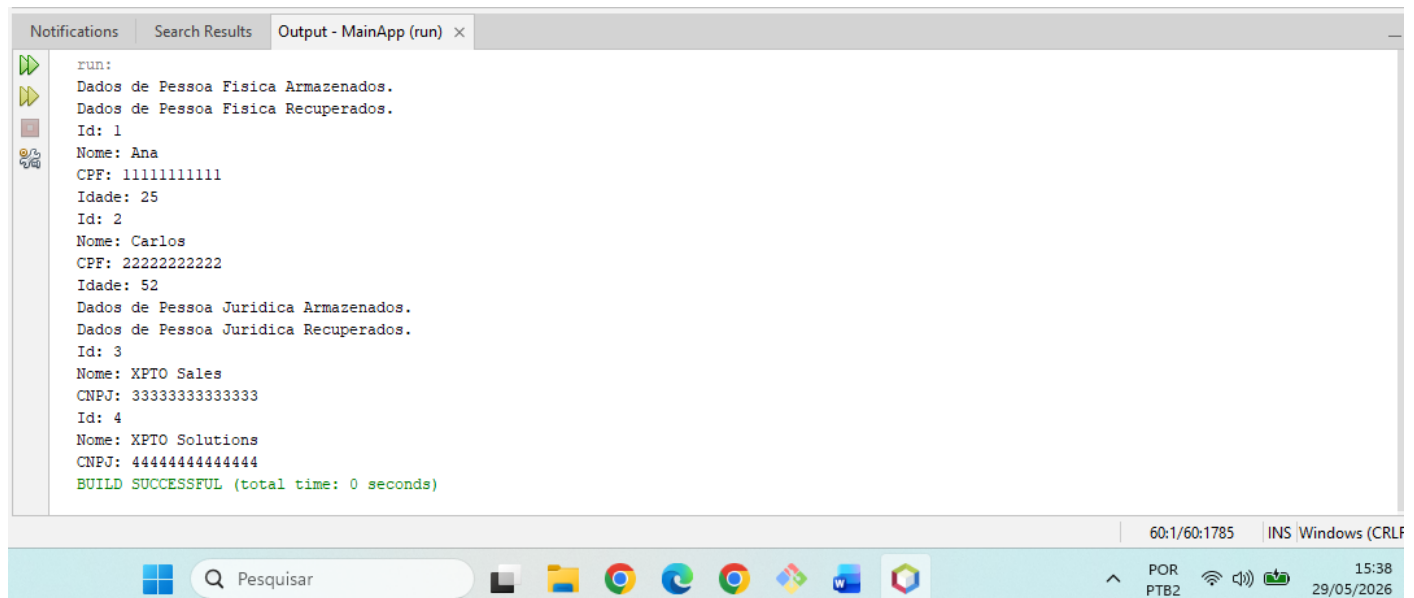
```

4. Os resultados da execução dos códigos também devem ser apresentados

Executando a aplicação



Resultado da execução



5. Análise e Conclusão

a. Quais as vantagens e desvantagens do uso de herança?

A herança permite que uma classe herde atributos e métodos de outra classe.

Vantagens:

- **Reutilização:** Aproveita código existente.
- **Polimorfismo:** Permite tratar objetos diferentes uniformemente.
- **Manutenção:** Centraliza correções na superclasse.
- **Organização:** Cria hierarquias lógicas claras.

Desvantagens:

- **Acoplamento:** Une rigidamente as classes.
- **Inflexibilidade:** Mudanças na superclasse afetam subclasses.
- **Complexidade:** Hierarquias longas dificultam leitura.
- **Fragilidade:** Quebra o encapsulamento da subclasse.

b. Por que a interface Serializable é necessária ao efetuar persistência em arquivos binários?

A interface Serializable serve como uma **marcação (marker interface)** para o compilador Java.

- **Autorização:** Sinaliza que o objeto pode ser convertido em fluxo de bytes.
- **Preservação:** Garante a reconstrução exata do estado do objeto.
- **Segurança:** Impede a exportação acidental de dados sensíveis do sistema.
- **Identificação:** Define o serialVersionUID para validar versões da classe.

c. Como o paradigma funcional é utilizado pela API stream no Java?

A API Stream utiliza o paradigma funcional para processar coleções de forma declarativa.

- **Expressões Lambda:** Passam comportamentos diretamente como argumentos de métodos.

- **Imutabilidade:** Não altera a fonte de dados original durante o processo.
- **Funções Puras:** Evita efeitos colaterais em operações de mapeamento e filtragem.
- **Avaliação Tardia:** Executa os filtros apenas quando uma operação terminal é chamada.
- **Interface Funcional:** Usa tipos padrão como Predicate, Function e Consumer.

d. Quando trabalhamos com Java, qual padrão de desenvolvimento é adotado na persistência de dados em arquivos?

O padrão de desenvolvimento mais adotado em Java para essa finalidade é o **DAO (Data Access Object)**.

- **Isolamento:** Separa a lógica de negócios das regras de armazenamento.
- **Abstração:** Oculta os detalhes técnicos de leitura e escrita do arquivo.
- **Interface:** Define contratos claros para operações de Criar, Ler, Atualizar e Deletar (CRUD).
- **Flexibilidade:** Facilita a troca do arquivo binário por um banco de dados no futuro.